

Apr 29, 26 17:29

main.cpp

Page 1/8

```

#include <avr/io.h>
#include <stdlib.h>
#include <avr/interrupt.h>

#define bitSet(reg, n)    ((reg) |= (1 << (n)))
#define bitClear(reg, n) ((reg) &= ~(1 << (n)))
#define bitCheck(reg, n) ((reg) >> (n) & 1)

// Prescaler need to be set to a value for required Fpwm such that
// TOP doesn't exceed its range. Prescaler = 128. See
// notes:26/4/2026 pg1.
//
// #define start_tc0      (TCCR0B |= 0b001) // prescaler = 1
// #define stop_tc0       (TCCR0B &= ~0b111)
// #define start_tc0      (TCCR0B |= 0b011) // prescaler = 64
// #define stop_tc0       (TCCR0B &= ~0b111)
//
// #define start_tc2      (TCCR2B |= 0b101) // prescaler = 128
// #define stop_tc2       (TCCR2B &= ~0b111)

// Timer/Counter overflow interrupt enable and disable
#define enable_tc2_interrupt (bitSet(TIMSK2, TOIE2))
#define disable_tc2_interrupt (bitClear(TIMSK2, TOIE2))

#define pin_oc0b    PD5 // for atmega328p
#define pin_oc2b    PD3 // for atmega328p

#define prescalerTC0 64
#define prescalerTC2 128

// Total_time_taken for the counter to count from 0->TOP-1 again.
// For the fast PWM TOP signal.
float timer2_overflow_time_us;

// Total_time_taken for the counter to count from 0->TOP-1->0. For
// the phase correct PWM TOP signal.
float timer0_overflow_time_us;

// T=1/F, the period of each clock tick. Where is F/prescaler.
// float clock_tc0;
float period_of_tick;

// Kai's original code, causes my_delay_us() to hang as num_timer2_overflow over
// flows:
// volatile unsigned int num_timer2_overflow = 0;
// volatile unsigned int num_timer2_overflow_max_sonar = 0;
//
// Avoids counter overflow in my_delay_us() for 1s delay.
volatile uint32_t num_timer2_overflows = 0;

// volatile uint8_t *ddr_port_d = &DDRD;
// volatile uint8_t *port_d = &PORTD;

// My TOP value. That is, how high the timer counter needs to count
// before one PWM cycle.
volatile uint8_t myTOPtc0;
volatile uint8_t myTOPtc2;

```

Apr 29, 26 17:29

main.cpp

Page 2/8

```

// Function Prototypes

// Debugging USART functions
void usart_debugging(void);
void usart_init(float baud_rate);
void usart_flush(void);
void usart_read_string(char *ptr_to_str);
void usart_send_byte(unsigned char data);
void usart_send_string(const char *ptr_to_str);
void usart_send_num(float num, char num_int, char num_decimal);

float linear_mapping(float x, float x1, float x2, float y1, float y2);
void my_delay_us(unsigned long x);
void config_tc0(void);
void config_tc2(void);

// Timer overflow, number of overflows for the current fast PWM TOP
// signal. A flag gets set in hardware then the ISR gets called.
//
ISR(TIMER2_OVF_vect)
{
    num_timer2_overflows++;
}

void config_tc0(void) {

    // Period of one tick
    period_of_tick = 1.0f/(float) (F_CPU/prescalerTC0);

    // Timer0 Phase Correct PWM, TOP = OCR0A (Mode 5)
    //
    // Set waveform generation mode: WGM02:0 = 101
    //
    // We want PWM output on OC0B in non-inverting mode. OCR0A will be
    // used for myTOP, or period of PWM frequency and OCROB for duty
    // cycle.
    //
    // We also require clear OC0B on compare match when up
    // counting. Set OC0B on compare match when counting down.
    //
    // Prescaler need to be set to a value for required Fpwm such that
    // TOP doesn't exceed its range. Prescaler=64. See
    // notes:26/4/2026 pg2.
    //
    // Also, setting the clock select bits, ie setting the prescaler
    // means the timers are started here, so really start_tc0 is what
    // we want to see correct output on oscilloscope.
    //
    TCCR0A = (1 << COM0B1) | (1 << WGM00);
    TCCR0B = (1 << WGM02);
    // TCCR0B = (1 << WGM02) | (1 << CS00) | (1 << CS01);

    // Setting TOP for phase correct PWM TOP Mode of TC0. From a
    // positive linear mapping of last three digits of my student id
    // card to TOP value range.
    //

```

Apr 29, 26 17:29

main.cpp

Page 3/8

```

// TCO is an 8 bit counter so it's range is from 0-255
//
// Check that myTOctc0=212
myTOPtc0 = (uint8_t)linear_mapping(1723, 1000, 2000, 100, 255);
usart_send_string("myTOPtc0=");
usart_send_num(myTOPtc0, 4, 0);
usart_send_byte('\n');

// OCROA is proportional to the Period/Frequency of the PWM
// signal(see equation for Fpwm signal) The larger the TOP value
// is it means the timer takes longer to complete one PWM cycle,
// so the frequency is lower. Conversely, the smaller the TOP
// means the timer completes one PWM faster, so the frequency is
// higher.
//
OCR0A = myTOPtc0;
OCR0B = 106; // This sets the Duty Cycle of the PWM signal on OCOB/PD5

// timer0_overflow_time_us=(total_number_of_ticks * period_of_tick) * 1.0e6
//
// In our phase correct PWM TOP mode we need to * 2 as the counter
// overflows after it counts from BOTTOM to TOP and back downwards
// to BOTTOM again, that is:
//
// 0 -> 1 -> 2 -> ... -> TOP -> ... -> 2 -> 1 -> 0
//
timer0_overflow_time_us = (float)myTOPtc0 * period_of_tick * 1.0e6 * 2.0f;
}

void config_tc2(void){

    period_of_tick = 1.0f/(float) (F_CPU/prescalerTC2);

    // Fast PWM, TOP = OCR2A (Mode 7)
    //
    // Set waveform generation mode: WGM02:0 = 111
    TCCR2A |= (1 << WGM21) | (1 << WGM20);
    TCCR2B |= (1 << WGM22);

    // We want PWM output on OC2B in non-inverting mode. OCR2A will be
    // used for myTOP, proportional period of PWM frequency and OCR2B
    // for duty cycle.
    //
    // We also require Clear OC0B on compare match. Set OC0B at
    // BOTTOM.
    //
    TCCR2A |= (1 << COM2B1);

    // Prescaler need to be set to a value for required Fpwm such that
    // TOP doesn't exceed its range. Prescaler = 128. See
    // notes:26/4/2026 pg1. However, we want start_tc2 to start the
    // timers very close together to get the desired effect on the
    // oscilloscope. Setting the prescaler here starts the counter
    // timing.
    //
    //TCCR2B |= (1 << CS22) | (1 << CS20);

```

Apr 29, 26 17:29

main.cpp

Page 4/8

```

// Setting TOP for phase correct PWM TOP Mode of TC0. From a
// positive linear mapping of last three digits of my student id
// card to TOP value range.
//
// TCO is an 8 bit counter so it's range is from 0-255
//
myTOPtc2 = (uint8_t)linear_mapping(1723, 1000, 2000, 100, 255);

// myTOPtc2 is = myTOctc0 - 1. This is the outcome for the
// TOP/myTOP value when you make Ftc0=Ftc2 as required. Refer to
// notes: 29/4/2026 pg3.
//
// OCROA is proportional to the Period/Frequency(see the
// calculation of the Fpwm signal) of the PWM signal. The larger
// the TOP value is it means the timer takes longer to complete
// one PWM cycle, so the frequency is lower. Conversely, the
// smaller the TOP means the timer completes one PWM faster, so
// the frequency is higher.
//
//
myTOPtc2--;
usart_send_string("myTOPtc2=");
usart_send_num(myTOPtc2, 4, 0);
usart_send_byte('\n');

OCR2A = myTOPtc2; // This sets the Period/Frequency of the PWM signal.
OCR2B = 106; // This sets the Duty Cycle of the PWM signal on OCOB/PD
5

// timer2_overflow_time_us=(total_number_of_ticks * period_of_tick) * 1.0e6
//
// In our fast PWM TOP mode the timer/counter counts as such:
//
// 0 -> 1 -> 2 -> ... -> TOP -> 0 -> 1 -> ...,
//
// Note that the the reset to 0 is part of the cycle, resulting in
// TOP+1 transitions for the period.
//
timer2_overflow_time_us = ((float)myTOPtc2 + 1.0f) * period_of_tick * 1.0e6;
}

int main(void)
{
    usart_init(9600);

    bitSet(DDRD, pin_oc0b); // PD5 as output
    bitSet(DDRD, pin_oc2b); // PD3 as output

    config_tc0(); // configure TC0, Timer Counter 0
    config_tc2(); // configure TC2, Timer Counter 2

    // They must be enabled quickly together to see the desired output
    // on the oscilloscope.
    start_tc0; // enable TC0
    start_tc2; // enable TC2
    sei();

```

Apr 29, 26 17:29

main.cpp

Page 5/8

```

while (1){

    // Continuously output on OCR0B(PD3, blue wire and trace) and
    // OCR2B(PD5, yellow wire and trace).

    // TC0 Phase Correct PWM, TOP = OCR0A (Mode 5)
    // TC2 Fast PWM, TOP = OCR2A (Mode 7)

    // causing problems with finding the same period as TCNT2 gets
    // reset there.
    //
    //my_delay_us(1000UL * 1000UL); // 1s delay
}

}

////////////////////////////////////
//                               User defined functions                               //
////////////////////////////////////

void my_delay_us(unsigned long x){

    // Delays, determined with a hardware timer/counter for x us.

    // NOTE: that a float assigned to an integer returns the integer
    // value with no decimal point values.

    // Number of complete Timer2 overflow periods needed.
    unsigned long num_complete_overflows = ( (float)x / timer2_overflow_time_us
);

    // Remaining time after complete overflows, in microseconds.
    float remainder_time_us =
        ( (float)x - ((float)num_complete_overflows*timer2_overflow_time_us) );

    // Convert remaining time into Timer2 ticks.
    unsigned char remainder_time_ticks =
        ( remainder_time_us / (period_of_tick * 1.0e6) );

    // Delay for complete number of overflows of Timer2
    if (num_complete_overflows > 0){
        num_timer2_overflows = 0;
        TCNT2 = 0;

run    // // TODO: Test Clears and pending overflow flag so the ISR doesn't
        // immediately and have the delay overflow too short.
        //TIFR2 = (1 << TOV2);

        enable_tc2_interrupt;
        while (num_timer2_overflows < num_complete_overflows);
        disable_tc2_interrupt;
    }

    // Delay for remainder number of ticks
    TCNT2 = 0;
    while (TCNT2 < remainder_time_ticks);
}

```

Friday May 08, 2026

main.cpp

Apr 29, 26 17:29

main.cpp

Page 6/8

```

}

float linear_mapping(float x, float x1, float x2, float y1, float y2){

    // Linear mapping equation: slope = (y2 - y1)/(x2 - x1)
    //
    // We have y vs x
    //
    // The slope is essentially the gradient of the straight line, also
    // known as the rate of change. The rate of change in a linear
    // equation is constant. So when we use linear mapping, we apply
    // a constant rate of change between the input range and the
    // output range.

    // To get greater y more quickly you need to increase the slope by
    // increasion y2.

    // By swapping y1 and y2 the slope becomes -ve(as y2>y1) so we get
    // a swapped around effect, or -ve linear mapping.

    // Clamp input so it stays in sensor range.
    //
    // 1. Clamp the input to the lower bound
    if (x <= x1) return y1;

    // 2. Clamp the input to the upper bound
    if (x >= x2) return y2;

    // Note: You lose precision with integer division because integers
    // cannot store fractional parts. When two integers are divided,
    // the result is truncated (the decimal part is discarded).

    // 3. Perform Linear Interpolation (Lerp)
    // Formula: y = y1 + (x - x1) * (slope)
    // where slope = (y2 - y1) / (x2 - x1)
    return y1 + (x - x1) * (y2 - y1) / (x2 - x1);
}

////////////////////////////////////
//                               USART helper functions                               //
////////////////////////////////////

void usart_init(float baud_rate){

    // Setting the baud rate register:
    //
    // Fosc, internal Crystal Arduino r3 = 16MHz
    // ubrr0 = (Fosc/prescale factor * baud rate) - 1
    // U2Xn=0, prescale factor is 2/4/2=16
    // U2Xn=1, prescale factor is 2/4=8

    float ubrr0 = ( (F_CPU / (16.0 * baud_rate)) - 1);
    int ubrr0a = (int)ubrr0;

    // Round up if neccessary to the next integer.
    if (ubrr0 - ubrr0a >= 0.5) {
        ubrr0a = ubrr0a + 1;
    }
}

```

3/4

Apr 29, 26 17:29

main.cpp

Page 7/8

```

}

// Set baud rate
UBRR0 = ubrr0a;

// Enable transmitter and receiver, and the Interrupt Service
// Routine on the RX buffer.
UCSR0B = (1 << TXEN0) | (1 << RXEN0) | (1 << RXCIE0);

// Set frame format: 8 bit data, 1 stop bit, no parity
UCSR0C = (0 << UMSEL01) | (0 << UMSEL00) | // Async mode
          (0 << UPM01) | (0 << UPM00) | // No parity
          (0 << USBS0) | // 1 stop bit
          (1 << UCSZ01) | (1 << UCSZ00); // 8-bit character
}

void usart_flush(void){
    char dummy;

    // Check if the UCSROA(USART Control and Status Register) has the
    // RXC0 flag set. If flag set then there is unread data in the
    // receive buffer. Read what's in the RX buffer which clears the
    // buffer.
    while (bitCheck(UCSR0A, RXC0))
        dummy = UDR0;

    ((void)dummy); // To suppress warning
}

void usart_send_byte(unsigned char data){

    // Send a byte when the UDRE0(USART data register empty flag) id
    // is set. In the UCSROA (UART contorl or status register A). It
    // means that the USART data register is empty.

    while (!bitCheck(UCSR0A, UDRE0))
        ;
    UDR0 = data;
}

// check if const is really needed, cgpt says c++ compiler is strictor
// than c compiler.
void usart_send_string(const char *pstr){

    // note: const means a read-only-string, prevents accidentally
    // modifying string literals(which causes crashes). String
    // literals in c are a sequence of characters enclosed in double
    // quotes, that is, an constant array of characters that are null
    // terminated, ie '\0' terminated.

    // Send each byte, one at a time, till the string terminator '\0'
    // is sent. '\0' is literally the numeric value 0 written in
    // character form(NULL ACSII character).
    while (*pstr != '\0') {
        usart_send_byte(*pstr);
    }
}

```

Apr 29, 26 17:29

main.cpp

Page 8/8

```

        pstr++;
    }
}

void usart_send_num(float num, char num_int, char num_decimal){

    // Send a number from MCU to PC by converting it to a string.

    // dtostrf(num, width, precision, string) converts float to an
    // ASCII string. We add a string terminator '\0' so the PC knows
    // when we are at the end of a string).

    char str[20];
    if (num_decimal == 0)
        dtostrf(num, num_int, num_decimal, str);
    else
        dtostrf(num, (num_int+num_decimal+1), num_decimal, str);
    str[num_int+num_decimal+1] = '\0';
    usart_send_string(str);
}

```